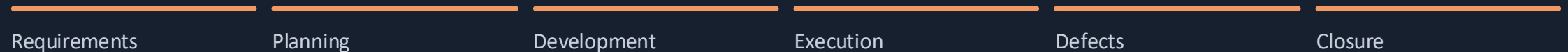


The Software Testing Process

From requirements to release: how testing is planned, built, run and managed, and who owns quality.



AGENDA

What we'll cover today

01

Why a testing process

Seven principles and the lifecycle

04

Test development

Design techniques, test cases

07

Closure & challenges

Exit criteria, common pitfalls

02

Requirement analysis

Testable requirements, traceability

05

Test execution

Running, recording, metrics

08

Tester vs QA vs QC

Who does what, and why it matters

03

Test planning

Scope, approach, levels and types

06

Defect management

Lifecycle, reports, severity vs priority

09

Hands-on exercise

Test a login page together

Seven principles behind every test process

Testing shows presence of defects

It can prove bugs exist, never that none remain.

Exhaustive testing is impossible

Use risk and test techniques to decide what to test.

Early testing saves time and money

Catch defects in requirements and design, not in production.

Defects cluster together

A few modules usually hold most of the defects.

Beware the pesticide paradox

The same tests, repeated, stop finding new bugs. Refresh them.

Testing is context dependent

A banking app and a mobile game need different approaches.

Absence of errors is a fallacy

Bug-free software that misses user needs still fails.

A test process turns these into decisions: **where** to test, **how much**, and **when**.

The testing lifecycle at a glance



Testing starts before any code exists

Testers review requirements to find gaps, ambiguity and untestable statements. The cheapest defect to fix is one caught on paper.

Entry: requirement docs or user stories available (BRD, SRS, backlog)

Exit: questions resolved, RTM drafted, automation candidates noted

Study the requirements

Functional and non-functional: performance, security, usability.

Ask questions

Clarify ambiguity with the BA, product owner and developers.

Identify what is testable

Agree acceptance criteria for every item.

Build the traceability matrix

Map every requirement to the tests that will cover it.

Spot automation candidates

Stable, repetitive, high-value flows.

Make every requirement testable

VAGUE

“The search page should load fast.
fast.”

Fast for whom? Measured how? Under what load? Nobody can say whether it passed.

TESTABLE

“Search results appear within 2 seconds
seconds for 95% of requests with 500
500 concurrent users.”

A number, a condition and a measure: pass or fail is objective.

A good requirement is:

Clear

Complete

Consistent

Measurable

Unambiguous

Traceable

The Requirement Traceability Matrix

Links each requirement to its tests and defects, so nothing goes untested and every change shows its impact.

Req ID	Requirement	Test cases	Status	Defects
REQ-01	User logs in with email and password	TC-001–003	Passed	—
REQ-02	Account locks after 5 failed logins	TC-004, TC-005	Failed	BUG-112
REQ-03	User resets password via email link	TC-006, TC-007	In progress	—
REQ-04	Session expires after 30 min idle	None	Not covered	—

Illustrative example. REQ-04 has no test cases: the matrix makes that coverage gap visible before release.

What a test plan must answer



Scope

What is in and out: features, platforms, integrations.



Approach

Test levels, types and techniques; manual vs automated.



Resources and roles

Who tests what; the skills and tools needed.



Schedule

Milestones and effort estimates tied to the release plan.



Environment and data

Hardware, software, accounts and test data required.



Entry and exit criteria

When testing can start, and when it is done.



Risks and mitigation

What could go wrong, and the contingency.



Deliverables

Test cases, reports, metrics and sign-off.

Choosing test levels and types

Level	What is tested	Usually by	Example
Unit	A single function or class in isolation	Developers	calculateTax() returns the right amount
Integration	Interfaces between modules or services	Developers and testers	Order service calls the payment API correctly
System	The complete application, end to end	Test team	Place an order from login to confirmation
Acceptance (UAT)	Fitness for business use	Users, product owner	Finance team signs off the month-end report

Test types that apply across levels

Functional

Smoke and sanity

Regression

Performance

Security

Usability

Compatibility

Blue: functional (what the system does). Orange: non-functional (how well it does it).

Use techniques, not guesswork

SPECIFICATION-BASED

Black-box

Derived from requirements; no knowledge of the code.

- Equivalence partitioning
- Boundary value analysis
- Decision tables
- State transition
- Use case testing

STRUCTURE-BASED

White-box

Derived from the code's internal logic and paths.

- Statement coverage
- Branch coverage
- Condition coverage
- Path coverage

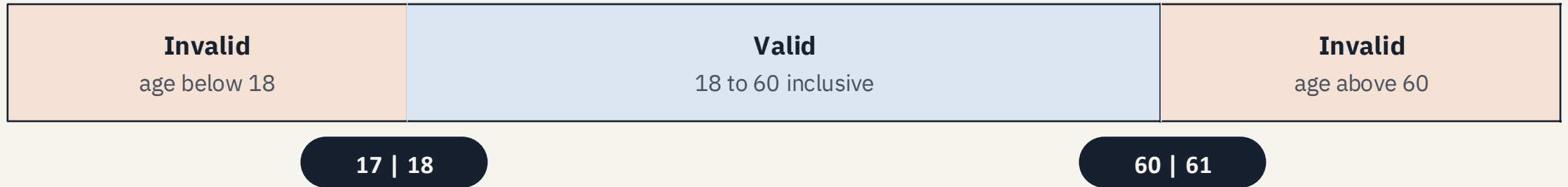
EXPERIENCE-BASED

Tester's judgement

Uses knowledge of past defects and the domain.

- Error guessing
- Exploratory testing
- Checklist-based testing

Example: an age field that accepts 18 to 60



Equivalence partitioning

One value per partition is enough: 10, 35 and 75. **3 tests.**

Boundary value analysis

Defects hide at the edges: 17, 18, 60 and 61. **4 tests.**

Also add invalid partitions for empty, negative, decimal and non-numeric input.

Anatomy of a good test case

Good test cases are

Atomic

One objective per test case.

Repeatable

Anyone who runs it gets the same result.

Traceable

Linked to the requirement it verifies.

Independent

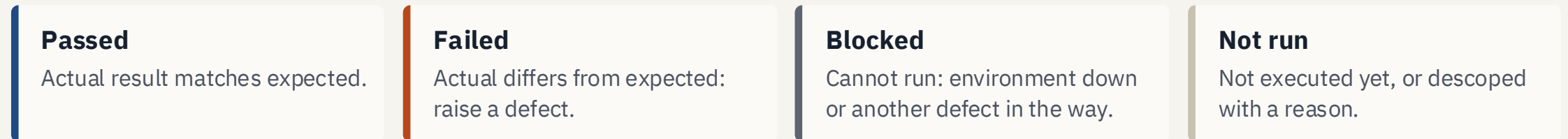
No hidden dependency on another test's order.

ID	TC-004 · traces to REQ-02 · Priority: High
Title	Account locks after 5 failed logins
Precondition	Active account user@test.com exists and is not locked
Steps	1. Open the login page 2. Enter the email with a wrong password, submit 3. Repeat step 2 until 5 attempts are made 4. Enter the correct password, submit
Test data	user@test.com · wrong password: WrongPass!1
Expected	After attempt 5 an “Account locked” message shows; in step 4 the correct password is also rejected

Run, record, report: every result is evidence



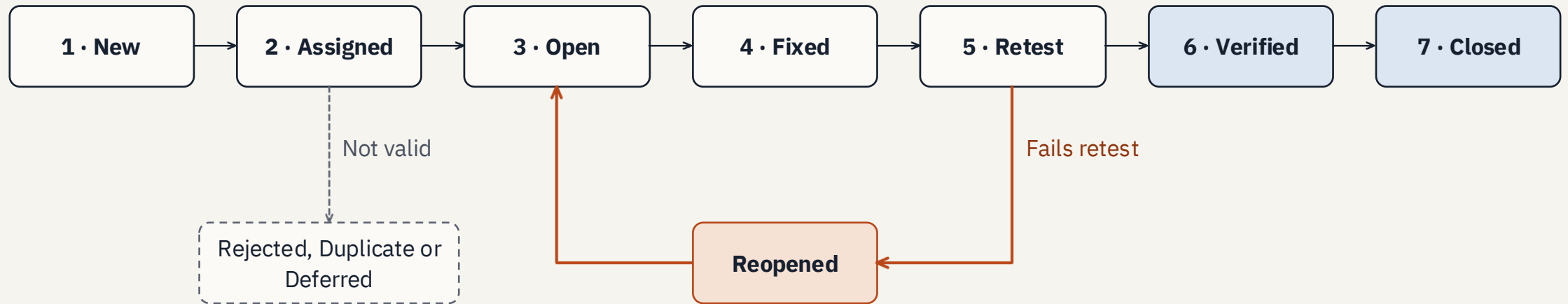
Every executed test gets one status



Metrics that show where testing stands

Metric	How to calculate	What it tells you
Execution progress	$\text{executed} \div \text{planned test cases} \times 100$	Are we on schedule?
Pass rate	$\text{passed} \div \text{executed} \times 100$	How stable is the build?
Defect density	$\text{defects} \div \text{size (KLOC, story points or modules)}$	Which areas are riskiest?
Open defects by severity	unresolved defects, grouped by severity	Are we ready to release?
Defect leakage	$\text{defects found after release} \div \text{all defects} \times 100$	How effective was testing?

The life of a defect



Tester

Raises it, retests the fix, then verifies, closes or reopens.

Test lead or PM

Triages: assigns, defers or rejects it.

Developer

Opens it, fixes it and hands it back for retest.

Write bug reports developers can act on

Every report needs

- A clear title: what fails, where
- Minimal steps to reproduce
- Expected vs actual result
- Environment and build
- Evidence: screenshot, video, logs
- Severity and priority
- One defect per report

BUG-112 · Account not locked after 5 failed logins

Environment	Staging, build 2.4.1 · Chrome on Windows 11
Steps	1. Open the login page 2. Log in as user@test.com with a wrong password 3. Repeat step 2 six times
Expected	Account locks after attempt 5 (REQ-02)
Actual	Attempt 6 still shows “Invalid password”; the account never locks
Severity	High · Priority: High
Evidence	Screen recording, server log extract · found by TC-004

Severity is impact. Priority is urgency.

Severity

How badly the defect affects the system.
Usually set by the tester.

Priority

How soon it must be fixed. Usually set by
the product owner or lead.

The two are independent, so every
defect needs both.

HIGH SEVERITY · HIGH PRIORITY

Checkout crashes when paying by card.

Fix now.

HIGH SEVERITY · LOW PRIORITY

App crashes in a yearly report used by
one admin.

Fix in a later release.

LOW SEVERITY · HIGH PRIORITY

Company name misspelled on the home
page.

Fix before release.

LOW SEVERITY · LOW PRIORITY

Typo in an old help article.

Fix when convenient.

Close the loop before the next release

Closure turns one test cycle into knowledge for the next.

01

Check exit criteria

Planned tests executed, no open critical defects, coverage target met. Then sign-off.

02

Write the summary report

What was tested, results and metrics, open risks, and a go or no-go recommendation.

03

Learn and archive

Hold a retrospective, archive test cases, data and scripts, and grow the regression suite.

CHALLENGES

Common challenges, and what helps

Unclear or changing requirements

Tests written against yesterday's spec.

Helps: early reviews, agreed acceptance criteria, a live RTM.

Tight deadlines

Testing squeezed into the last days of the sprint.

Helps: risk-based prioritisation, testing from day one.

Unstable environments and data

Blocked runs; “works on my machine”.

Helps: dedicated environments, scripted setup, managed test data.

A growing regression suite

Every release needs more re-testing.

Helps: automate stable, high-value paths; prune stale tests.

Flaky automated tests

Random failures erode trust in results.

Helps: quarantine, fix the root cause, avoid fixed waits.

Dev and test working apart

Defects bounce back as “cannot reproduce”.

Helps: three-amigos sessions, clear bug reports, shared tools.

Tester vs QA vs QC

QUALITY ASSURANCE

The process · prevents defects

QUALITY CONTROL

The product · detects defects

TESTING

The activity · runs the checks

QA: quality assurance

Builds quality into how work is done: standards, reviews, audits, training, process improvement.

“Does our process prevent defects?”

QC: quality control

Checks finished work against the standard: inspections, deliverable reviews, testing.

“Does this product meet the requirements?”

Tester

Carries out the testing part of QC: designs and runs tests, reports defects.

“Does this feature behave as expected?”

QA, QC and testing side by side

Aspect	Quality assurance	Quality control	Testing
Focus	The process	The product	The product's behaviour
Nature	Preventive, proactive	Detective, reactive	Detective, reactive
When	Throughout the lifecycle	After a deliverable is produced	During test execution
Goal	Stop defects being introduced	Find defects before release	Find failures, confirm fixes
Owned by	Whole team, led by QA or process leads	QC team, reviewers, testers	Testers (developers for unit tests)
Examples	Standards, audits, training, retrospectives	Inspections, walkthroughs, testing	Test cases, test runs, bug reports

In job titles the lines blur: a “QA engineer” often does all three. What matters is the focus: the process or the product.

HANDS-ON EXERCISE

Your turn: test a login page

THE REQUIREMENT

Users log in with email and password. Passwords are 8 to 20 characters. After 5 failed attempts, the account locks for 15 minutes.

01

Question it

List three questions for the product owner.

02

Partition it

Find the partitions and boundaries for password length.

03

Write it

Two test cases with steps and expected results.

04

Report it

A bug report for: “Login button does nothing on Safari”.

WRAP-UP

Five things to take back to your team

- 01 Testing is a lifecycle, not a phase at the end.
- 02 Testable requirements and a clear plan prevent most surprises.
- 03 Techniques beat guesswork: partitions, boundaries, decision tables.
- 04 Good bug reports and simple metrics turn results into decisions.
- 05 QA improves the process; QC and testing check the product.

THANK YOU

Questions and discussion

Which part of this process does your team do well today, and which part would you strengthen first?

[Presenter name] · [Contact email]